

MLP Memoization

➤ Introduction

A technique used to **save computation time** by remembering results that have already been calculated.

It's not limited to ML, it's a *general programming optimization*, but when applied to **neural networks**, especially **MLPs**, it can drastically improve efficiency in forward and backward passes.

➤ What is Memoization?

Definition:

Memoization is a method of **caching (storing)** the results of expensive function calls and returning the cached result when the *same inputs* occur again.

In simple terms:

“Don't repeat work you've already done.”

It's like remembering answers to problems you've already solved so you don't have to solve them again.

➤ Example:

Say you compute $f(5)$ for Fibonacci — you store it.

Next time you need $f(5)$, you just **reuse the stored result**, not recompute it.

This turns **recursive, redundant computation** into **efficient lookup**.

➤ Fibonacci Sequence

The Fibonacci sequence is a classic way to demonstrate memoization.

✓ Without Memoization

```
def fib(n):  
    if n <= 1:  
        return n  
    return fib(n-1) + fib(n-2)  
  
print(fib(35))
```

This will take **a long time**, because every call recomputes smaller Fibonacci numbers repeatedly.

For example, $\text{fib}(5)$ computes $\text{fib}(4)$ and $\text{fib}(3)$
and $\text{fib}(4)$ again computes $\text{fib}(3)$ and $\text{fib}(2)$ → repeated work!

✓ With Memoization

```
memo = {}
```

```
def fib_memo(n):
    if n in memo:
        return memo[n]
    if n <= 1:
        memo[n] = n
    else:
        memo[n] = fib_memo(n-1) + fib_memo(n-2)
    return memo[n]

print(fib_memo(35))
```

Now, each value is computed **once** and stored in the dictionary memo.
 Future calls reuse the result → **massive speed-up**

✓ MLP Memoization

Now, let's connect this to **MLPs (Multilayer Perceptrons)** in deep learning.

The Problem:

In MLPs, during training:

- Many computations (especially during **forward propagation**) are **repeated** for similar inputs or during **backward passes**.
- For large networks, this becomes **computationally expensive**.

The Idea:

Use **memoization** to **store intermediate results** activations, gradients, or function outputs that are **reused** later.

✓ Example:

In forward propagation:

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$$

$$a^{(l)} = f(z^{(l)})$$

If a particular ($a^{(l-1)}$ or ($z^{(l)}$) has been computed before (e.g., same input batch or reused during multiple passes), you can store and reuse it.

Same in **backpropagation**:

$$\delta^{(l)} = (W^{(l+1)})^T \delta^{(l+1)} \cdot f'(z^{(l)})$$

If ($f'(z^{(l)})$) was computed once, no need to recompute it.

➤ Where Memoization Helps in MLPs:

Area	What's Cached	Why It Helps
Forward Pass	Activations $a^{(l)}$, Linear Outputs $z^{(l)}$	Avoids redundant computation for repeated inputs
Backward Pass	Gradients, partial derivatives	Reduces redundant gradient evaluations
Loss Calculation	Same input patterns or samples	Saves time when loss computed multiple times
Batch Processing	Common sub-results reused across mini-batches	Enhances performance in recurrent calls

➤ Analogy:

Think of it like caching in browsers

When you revisit a website, the browser doesn't download everything again; it uses stored files for faster loading.

MLPs do something similar with memoization:

Cache repeated math → reduce computation → train faster.

➤ Benefits

1. Reduces computation time
2. Avoids redundant matrix operations
3. Improves training efficiency (especially in large models or repeated mini-batches)
4. Can be combined with other optimizations like gradient checkpointing

➤ Trade-offs

Advantage	Drawback
Faster computation	More memory usage (storing cached values)
Efficient reuse	Cache invalidation must be handled properly
Easier debugging	Slightly more complex code logic

➤ Outro

To summarize:

- **Memoization** = Remembering previously computed results.
- **In MLPs**, it helps reuse repeated computations in forward & backward passes.
- It's a **smart optimization** technique for computational efficiency, especially in deep learning frameworks.

In libraries like **TensorFlow**, **PyTorch**, and **JAX**, memoization concepts are embedded in **computational graphs** — where the framework automatically remembers intermediate results to optimize training.

➤ **Key Takeaway**

“Memoization turns computation-heavy neural network operations into efficient lookups saving time, not accuracy.”
